

Image Processing Algorithms from Scratch Using Python

Author: Justin Ogle

This project demonstrates the implementation of fundamental image processing algorithms using Python and minimal external dependencies. Core techniques including image resizing, Gaussian filtering, edge-preserving smoothing, and image quality assessment are implemented to provide insight into the mathematical foundations of computer vision systems.

Image Resizing Fundamentals

This section demonstrates image resizing operations commonly used in computer vision and deep learning pipelines. The implementation focuses on image loading, dimension standardization, and image preparation workflows.

```
In [46]: from PIL import Image
from io import BytesIO
# Function to resize image
def resize_image(image, size):
    return image.resize(size)
# Load the Lena image
lena_path = "Lena.png"
with open(lena_path, "rb") as file:
    lena_data = file.read()
# Open image from bytes
lena_image = Image.open(BytesIO(lena_data))
# Resize the image to (224, 224)
resized_lena = resize_image(lena_image, (224, 224))
# Save the resized image
with open("resized_lena.jpg", "wb") as output_file:
    resized_lena.save(output_file)
# Display the resized image (not recommended without using external libraries)
resized_lena.show()
# Output message
print("Image resized and saved as 'resized_lena.jpg'.")
```

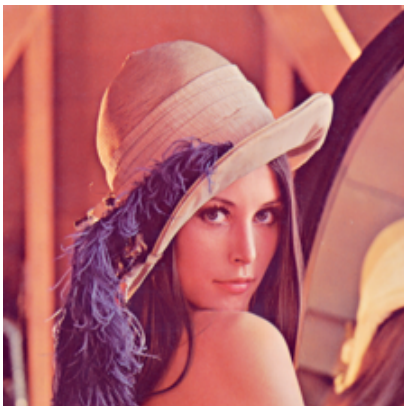


Image resized and saved as 'resized_lena.jpg'.

```
In [47]: from PIL import Image
from io import BytesIO
import matplotlib.pyplot as plt
# Function to resize image
def resize_image(image, size):
    return image.resize(size)
# Load the Lena image
lena_path = "Lena.png"
with open(lena_path, "rb") as file:
    lena_data = file.read()
# Open image from bytes
lena_image = Image.open(BytesIO(lena_data))
# Resize the image to (224, 224)
resized_lena = resize_image(lena_image, (224, 224))
# Save the resized image
with open("resized_lena.jpg", "wb") as output_file:
    resized_lena.save(output_file)
# Display the resized image using matplotlib
plt.imshow(resized_lena)
plt.axis('off') # Hide axis labels
plt.show()
# Output message
print("Image resized and saved as 'resized_lena.jpg'.")
```



Image resized and saved as 'resized_lena.jpg'.

Gaussian Noise Modeling and Filtering

This section investigates the effects of Gaussian noise and evaluates Gaussian filtering techniques using manually generated convolution kernels.

Kernel sizes evaluated:

- 5×5
- 7×7
- 9×9

- 11×11

The results illustrate the tradeoff between noise suppression and image detail preservation.

```
In [48]: from PIL import Image
from io import BytesIO
import numpy as np
from scipy.ndimage import convolve
# Load the Lena image (assuming Lena.png is in the same directory as the script)
with open("Lena.png", "rb") as file:
    lena_data = file.read()
# Add random noise to the image
def add_random_noise(image_array, noise_factor=0.1):
    noise = np.random.normal(0, noise_factor, image_array.shape)
    noisy_image_array = image_array + noise
    noisy_image_array = np.clip(noisy_image_array, 0, 255)
    return np.uint8(noisy_image_array)
# Apply Gaussian filter to the image
def apply_gaussian_filter(image_array, filter_size):
    kernel = np.fromfunction(
        lambda x, y:
            (1/(2*np.pi*filter_size**2))
            * np.exp(
                -(
                    (x-filter_size//2)**2
                    + (y-filter_size//2)**2
                )/(2*filter_size**2)
            ),
        (filter_size, filter_size)
    )

    # Normalize the kernel
    kernel = kernel / kernel.sum()

    filtered_image_array = np.zeros_like(image_array)

    for channel in range(filtered_image_array.shape[2]):
        filtered_image_array[:, :, channel] = convolve(
            image_array[:, :, channel],
            kernel,
            mode='constant',
            cval=0.0
        )

    return np.uint8(filtered_image_array)
# Load Lena image from bytes
lena_image = np.array(Image.open(BytesIO(lena_data)))
# Apply Gaussian filters of sizes 5x5, 7x7, 9x9, and 11x11
filter_sizes = [5, 7, 9, 11]
for size in filter_sizes:
    # Add random noise
    noisy_lena_image = add_random_noise(lena_image)

    # Apply Gaussian filter
    filtered_lena_image = apply_gaussian_filter(noisy_lena_image, size)

# Display noisy and filtered images
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
```

```
plt.subplot(1, 2, 1)
plt.imshow(noisy_lena_image)
plt.title("Noisy Lena Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(filtered_lena_image)
plt.title(f"Gaussian Filter {size}x{size}")
plt.axis("off")

plt.tight_layout()
plt.show()

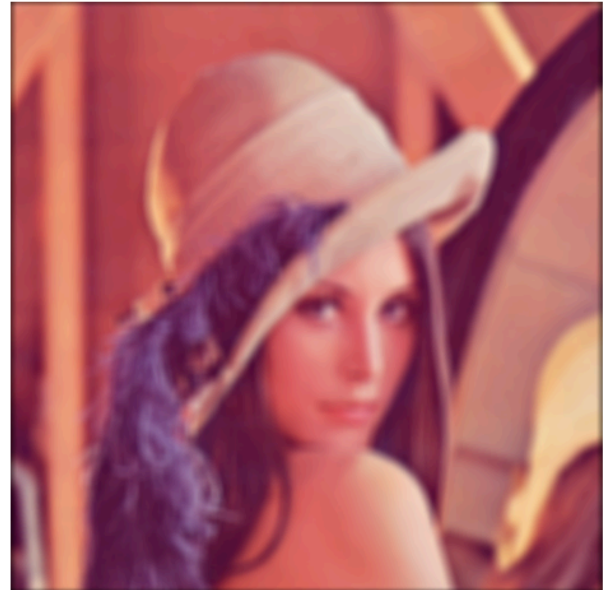
# Save processed images
Image.fromarray(noisy_lena_image).save(
    f"noisy_lena_{size}x{size}.jpg"
)

Image.fromarray(filtered_lena_image).save(
    f"gaussian_filter_{size}x{size}.jpg"
)
```

Noisy Lena Image



Gaussian Filter 11x11



Edge-Preserving Image Smoothing

This section evaluates Bilateral Filtering and a simplified Nagao-Matsuyama-inspired adaptive smoothing filter for image noise reduction.

The objective is to demonstrate edge-preserving filtering techniques that reduce noise while maintaining important image structures and boundaries.

Results are presented for algorithmic demonstration and qualitative comparison of filtering performance.

```
In [ ]: from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

# -----
```

```

# Bilateral Filter
# -----
def bilateral_filter(image, d, sigma_color, sigma_space):

    if len(image.shape) == 2:
        image = np.stack((image, image, image), axis=-1)

    result = image.copy()

    height, width, _ = image.shape

    for i in range(height):
        for j in range(width):

            pixel = image[i, j]

            i_min = max(0, i - d)
            i_max = min(height - 1, i + d)

            j_min = max(0, j - d)
            j_max = min(width - 1, j + d)

            for k in range(3):

                numerator = 0.0
                denominator = 0.0

                for x in range(i_min, i_max + 1):
                    for y in range(j_min, j_max + 1):

                        neighbor_pixel = image[x, y]

                        spatial_weight = np.exp(
                            -((x - i)**2 + (y - j)**2)
                              / (2 * sigma_space**2)
                        )

                        color_weight = np.exp(
                            -np.linalg.norm(
                                neighbor_pixel - pixel
                            )**2
                              / (2 * sigma_color**2)
                        )

                        weight = spatial_weight * color_weight

                        numerator += weight * neighbor_pixel[k]
                        denominator += weight

                result[i, j, k] = int(numerator / denominator)

    return result

# -----
# Nagao-Matsuyama Filter
# -----
def nagao_matsuyama_filter(image):

    # Convert RGB to grayscale if needed
    if len(image.shape) == 3:
        image = np.mean(image, axis=-1)

```

```

image = image.astype(np.float32)

result = image.copy()

height, width = image.shape

for i in range(1, height - 1):
    for j in range(1, width - 1):

        neighborhood = image[
            i-1:i+2,
            j-1:j+2
        ]

        center_pixel = image[i, j]

        local_mean = np.mean(neighborhood)
        local_variance = np.var(neighborhood)

        # Preserve edges in high-variance regions
        if local_variance > 100:
            result[i, j] = center_pixel

        # Smooth Low-variance regions
        else:
            result[i, j] = local_mean

    return np.clip(result, 0, 255).astype(np.uint8)

# -----
# Load and Resize Test Image
# -----
image_path = "Fig0343(a)(skeleton_orig).tif"

image_pil = Image.open(image_path)

# Resize image to reduce execution time
image_pil = image_pil.resize((128, 128))

image = np.array(image_pil)

gray_image = np.array(
    image_pil.convert("L")
)

# -----
# Apply Filters
# -----
result_bilateral = bilateral_filter(
    image.copy(),
    d=5,
    sigma_color=50,
    sigma_space=50
)

result_nagao = nagao_matsuyama_filter(
    gray_image.copy()
)

```

```

# -----
# Save Results
# -----
Image.fromarray(
    result_bilateral.astype(np.uint8)
).save("result_bilateral.png")

Image.fromarray(
    result_nagao.astype(np.uint8)
).save("result_nagao_matsuyama.png")

# -----
# Display Results
# -----
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(gray_image, cmap="gray")
plt.title("Original")
plt.axis("off")

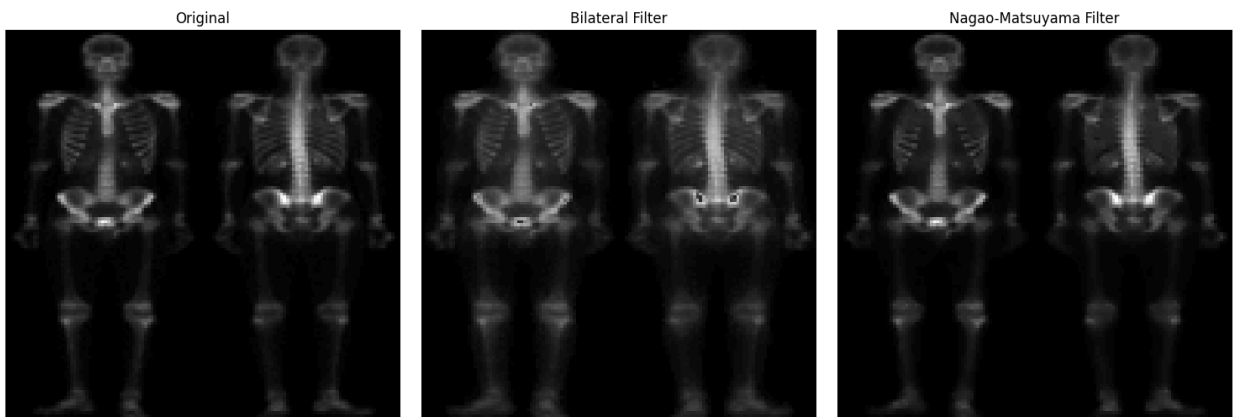
plt.subplot(1, 3, 2)
plt.imshow(result_bilateral.astype(np.uint8))
plt.title("Bilateral Filter")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(result_nagao, cmap="gray")
plt.title("Nagao-Matsuyama Filter")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Results saved:")
print(" - result_bilateral.png")
print(" - result_nagao_matsuyama.png")

```



Results saved:

- result_bilateral.png
- result_nagao_matsuyama.png

Image Quality Assessment

This section evaluates the impact of Gaussian noise on image quality using quantitative image assessment metrics implemented from scratch.

Metrics evaluated include:

- Root Mean Square Error (RMSE)
- Peak Signal-to-Noise Ratio (PSNR)
- Entropy

These metrics provide insight into image fidelity, noise degradation, and information content. RMSE measures pixel-level reconstruction error, PSNR measures signal quality relative to noise, and Entropy quantifies the complexity and information content present within the image.

```
In [50]: import numpy as np

# Define reference image
original_image = np.array([
    [139, 144, 149, 153, 155, 155, 155, 155],
    [144, 151, 153, 156, 159, 156, 156, 156],
    [150, 155, 160, 163, 158, 156, 156, 156],
    [159, 161, 162, 160, 160, 159, 159, 159],
    [159, 160, 161, 162, 162, 155, 155, 155],
    [161, 161, 161, 161, 160, 157, 157, 157],
    [162, 162, 161, 163, 162, 157, 157, 157],
    [162, 162, 161, 161, 163, 158, 158, 158]
], dtype=np.float64)

# Add Gaussian noise
sigma = 5

noisy_image = original_image + np.random.normal(
    0,
    sigma,
    original_image.shape
)

# Clip values to valid image range
noisy_image = np.clip(noisy_image, 0, 255)

# -----
# Calculate RMSE from scratch
# -----
mse = np.mean((original_image - noisy_image) ** 2)
rmse = np.sqrt(mse)

# -----
# Calculate PSNR from scratch
# -----
max_pixel = 255.0

psnr = 20 * np.log10(max_pixel / rmse)

print(f"RMSE: {rmse:.4f}")
print(f"PSNR: {psnr:.4f} dB")
```

RMSE: 4.9007

PSNR: 34.3257 dB

```
In [51]: from scipy.stats import entropy
```



```
# Flatten the 8x8 image to a 1D array
flat_image = original_image.flatten()

# Calculate entropy
image_entropy = entropy(flat_image)

print(f"Entropy: {image_entropy}")
```

Entropy: 4.158436058490848

=====

Conclusion

This project explored several foundational image processing techniques, including image resizing, Gaussian noise modeling, Gaussian filtering, edge-preserving smoothing, and image quality assessment.

Multiple algorithms were implemented directly in Python to provide insight into the underlying mathematical principles used in image processing systems.

The results demonstrate that image filtering often involves a tradeoff between noise reduction and detail preservation. Larger Gaussian filter kernels provided stronger noise suppression but introduced increased image blurring and loss of fine image detail.

Edge-preserving techniques such as Bilateral Filtering and adaptive neighborhood-based smoothing reduced noise while maintaining important structural boundaries and image features.

Image quality metrics including RMSE, PSNR, and Entropy were implemented and evaluated to quantify image degradation and reconstruction quality.

Overall, this project demonstrates the importance of selecting appropriate filtering and enhancement techniques based on application requirements. The balance between noise reduction, feature preservation, computational complexity, and image quality remains a fundamental consideration in computer vision and image processing systems.

=====

